Individual Plan

Emil Karlsson — emilk2@kth.se

March 20, 2024

- Title: Kubernetes as a Virtual Machine Management System: A Comparative Study of KubeVirt and Traditional Approaches
- Student: Emil Karlsson emilk2@kth.se
- Examiner: Cyrille Artho
- Supervisor (KTH): Han Fu
- External supervisor (Net Insight): Robert Ödman robert.odman@netinsight.net

1 Background

Virtualization is a technique that either splits or combines computing resources into one or more operating environments by creating a software layer that surrounds or underlies existing hardware or software. This enables the creation of virtualized resources that are decoupled from the underlying hardware. Virtualization can occur in various forms including the virtualization of storage, networks and servers. Virtualizing networks and storage is achieved by creating an abstraction layer on top of the physical network and storage resources, respectively, to provide a more flexible and scalable environment for other resources to interact with. Server virtualization, on the other hand, is used to create virtual computing environments that are decoupled from the underlying computing resources. By combining virtualized resources into a single operating environment, virtualization allows heterogeneous hardware resources to be used homogeneously and thus it improves resource utilization and flexibility [1, 2, 3].

To enable server virtualization and manage virtual computing environments, a hypervisor is used. A hypervisor is a software layer that operates between the hardware and the operating system (OS) and is responsible for creating virtual computing environments called virtual machines (VMs) [3, 4, 5]. VMs are emulations of physical computers that are perceived as real computers by applications running inside them. They provide a complete and independent environment by virtualizing an OS, rendering them suitable for a wide range of applications, including legacy applications and applications that require a high level of isolation. For this reason, VMs have long been the core part of IT infrastructure, serving as foundational components in system architecture [1, 2].

In parallel with the evolution of VMs, containers have emerged as an alternative to VMs. Container virtualization is an OS-level virtualization technique that allows for running multiple isolated applications on a single OS instance. This means container virtualization does not require a hypervisor, and instead uses the features and capabilities of the OS kernel to create and manage containers. As such, containers are more lightweight, require less overhead, and have faster startup times than VMs [6]. This has made containers a popular choice for scenarios in which scaling and portability are key,

such as in microservices, which are core components in modern cloud-native applications [7]. Figure 1 visualizes the differences between how VMs and containers are virtualized, and how the virtualization layer is placed on a lower level for VMs than for containers.

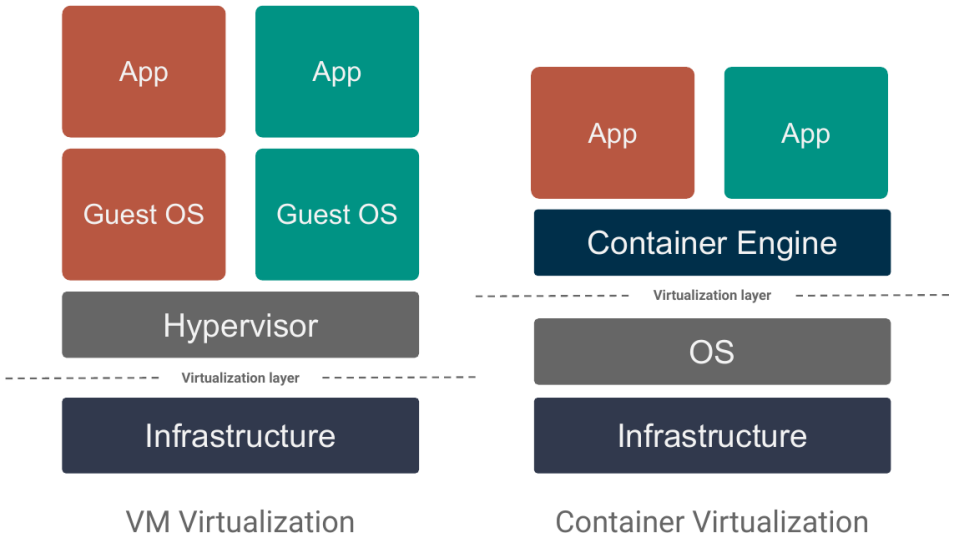

Figure 1: Overview of VMs and containers

However, while containers provide a lightweight and efficient alternative to VMs, they lack the deep isolation properties inherent to VMs. Sharing the OS kernel with the host and other containers opens up a larger attack surface, which implies that containers are more prone to security vulnerabilities. While there exist technologies that aim to unify the isolation benefits of VMs and the lightweight benefits of containers, such as Kata Containers, gVisor and Firecracker, it remains a fact that both containers and VMs have their use cases [8, 9].

Due to their different use cases, VMs and containers have traditionally been managed separately. VMs are often managed by a VM management system that provides a set of tools and processes to aid in the life-cycle management of VMs. This makes a VM management system act as a higher-level abstraction layer that unifies hypervisors, typically through an application programming interface (API) such as a command line interface (CLI) or a web-based user interface. Figure 2 depicts a broad overview of the role of a VM management system where clients interact with an interface to manage VMs that are distributed across several hosts. Moreover, a VM management system is often part of a larger infrastructure, such as Infrastructure as a Service (IaaS), that provides a wider range of virtualized resources including virtual networks and storage. Examples of such systems include OpenStack [10], CloudStack [11], and OpenNebula [12]. Containers, on the other hand, are often managed by a container orchestration platform. These platforms provide a similar set of tools and processes as a VM management system but for containers. Apart from creating and deploying containers, a container orchestration platform provides tools specifically designed to manage the life cycle of containers, such as scheduling and scaling. Examples of such platforms include Kubernetes [13], Nomad [14], and Docker Swarm [15], with the former being the most widely adopted. Furthermore, with the evolution and adaption of container orchestration platforms like Kubernetes, there has been an ongoing convergence in how VMs and containers are managed [16].

Kubernetes, originally designed to manage containerized applications at scale, has recently been extended to accommodate VMs through the KubeVirt project. KubeVirt is an API extension for Kubernetes, that enables the running of VMs inside a Kubernetes environment alongside containers,

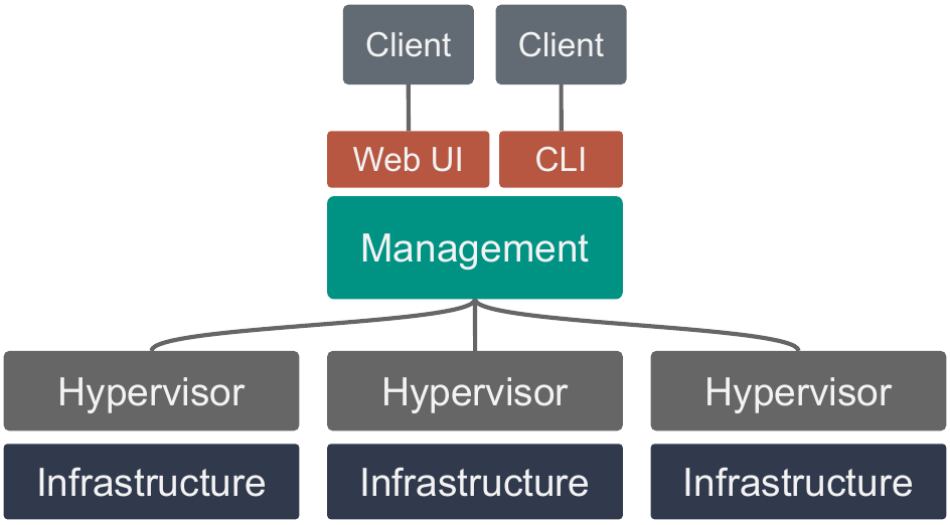

Figure 2: Overview of a VM management system

creating a common environment for both virtualization technologies [17]. Figure 3 depicts a high-level overview of how VMs and containers coincide in a Kubernetes environment using KubeVirt, where traffic flows natively between the two types of resources. Further technical details of how KubeVirt achieves this will be discussed in the report.

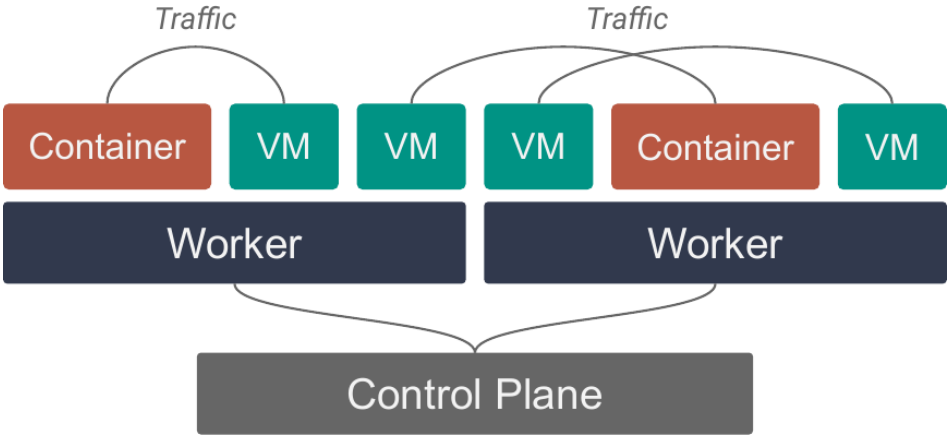

Figure 3: Overview of VMs and containers managed by using KubeVirt

The approach of incorporating VMs in Kubernetes not only benefits from the flexible, scalable and mature infrastructure that Kubernetes provides but also from tools and processes that are designed to integrate with or manage the life cycle of Kubernetes resources. This means by using KubeVirt, VMs can be managed similarly to containers, creating a more cloud-native approach to VM management [7]. Furthermore, consolidating the management of VMs and containers into a single platform could offer benefits in terms of reduced architectural complexity and simpler hardware management, but also practical aspects such as reduced required skill set in an organization [18, 19].

The integration of VM management into Kubernetes through KubeVirt represents a significant milestone toward a more agile, efficient, and unified IT infrastructure [19]. However, KubeVirt's practicality in a real-world scenario is yet to be fully assessed. As such, this project aims to evaluate Kubernetes with KubeVirt as a VM management system and compare it with traditional approaches

to understand its practicality and limitations.

2 Objective

This project aims to explore the integration of virtual machines into Kubernetes environments using the API extension KubeVirt. By comparing how virtual machines are managed using KubeVirt against traditional virtual machine management systems, this project aims to address possible inefficiencies and complexities that arise when managing virtual machines outside of Kubernetes. This includes performance comparisons with metrics such as virtual machine deployment time, deletion time, and CPU utilization during those operations. It also includes assessing available features, such as supported virtualization technologies, snapshot management tools, integration with external tools such as GitOps etc. Additionally, the comparison will include the perspective of a system administrator; How quickly can virtual machines be deployed? Finally, the comparisons will consider an application-to-application perspective, which includes addressing potential usability improvements when consolidating to a single API.

2.1 Case Study: Net Insight

This project is particularly relevant to Net Insight, the company collaborating on this project. Net Insight is a company specializing in live-streaming media, and they host their development environment on-premise using the virtual machine management system OpenNebula. Many of their use cases involve testing software in containers, but also in virtual machines when a completely isolated environment is required. Furthermore, allowing for well-adapted features in Kubernetes such as GitOps [20] could potentially improve the deployment streamline when testing their software. Finally, many of their developers, more accustomed to Kubernetes, could prefer a Kubernetes-based environment. Adopting Kubernetes with KubeVirt could thus enhance the developer experience and efficiency.

2.2 Case Study: kthcloud

The project is also of interest to the kthcloud project [21][22] at KTH Royal Institute of Technology, a private cloud offering computing and hosting resources for students and researchers. The kthcloud project faces challenges in how virtual machines are managed, but also in running virtual machines and containers in separate environments (CloudStack and Kubernetes). These challenges include separate APIs (imperative vs declarative REST APIs), inefficient use of hardware resources due to having two different resource schedulers and nested virtualization, and the difficulty in training new team members in two different systems.

3 Literature Study

3.1 Current challenges

The Cloud Native Computing Foundation (CNCF) presents in a report some of the challenges of managing VMs using a traditional IaaS platform and uses OpenStack as an example [18]. While OpenStack has been a popular choice for managing VMs, it has been criticized for being complex and difficult to manage, as well as having a steep learning curve. They argue that the complexity

of OpenStack is not always needed and that it is possible to manage VMs using a simpler and more efficient platform, such as Kubernetes with KubeVirt.

Additionally, the CNCF report highlights the importance of having a unified platform for managing both VMs and containers, as it could reduce the complexity of the underlying technology stack and management costs. These claims are further supported by a report by Kubacki [19]. He argues that managing VMs using traditional VM management systems alongside Kubernetes leads to unnecessary costs, such as maintaining personnel and skill sets for two technologies. Moreover, by allowing the coexistence of VMs and containers in a single platform, KubeVirt could potentially reduce any barrier of entry around managing VMs for organizations that are already using Kubernetes for container orchestration, or act as a catalyst for organizations considering adopting Kubernetes.

3.2 KubeVirt challenges

While KubeVirt is presented as a solution to the challenges of managing VMs using traditional VM management systems, it is important to consider the challenges of using KubeVirt itself. A study by Trakulmaiphol and Piromsopa presents some shortcomings when migrating from a traditional VM management system to KubeVirt [23]. One of the issues identified is the management of IP addresses, particularly when maintaining static IP addresses in a dynamic IP environment, such as Kubernetes. In the study, they developed a framework to solve some of the issues on the application level, such as DNS queries, and provided a set of guidelines related to KubeVirt's unique features, such as the Multus Container Network Interface.

A common practice when managing Kubernetes clusters is to deploy Kubernetes nodes on VMs created by a VM management system. This means that when using KubeVirt, VMs are created inside other VMs leading to nested virtualization. Ye et al. argue in a study that nested virtualization raises security concerns due to the introduction of a larger trusted computing base (TCB) [24]. By using a larger TCB, the size of the system's attack vector increases leading to an increased risk for vulnerabilities. For this reason, nested virtualization might not be suitable when strict security is required, such as applications managing sensitive or confidential data. This study is particularly relevant as it highlights a potential limitation if KubeVirt is simply seen as another Kubernetes resource, and not as a VM management system.

3.3 Usability of Kubernetes and KubeVirt

Using Kubernetes as a VM management system is a relatively new concept, and there is limited research on the usability of KubeVirt. However, several studies have evaluated the usability of Kubernetes, which is relevant as KubeVirt is an extension built on top of Kubernetes.

Kubernetes API is inherently declarative, meaning an approach that promotes the usage of documents that describe a desired state of a system. A study by Sonninen investigates many of the possible benefits of using a declarative approach in system architecture [25]. The study found that a declarative approach facilitates GitOps, a way to manage infrastructure using files in Git, which simplifies system and resource management and enables easier rollbacks. Using Git as a source of truth for the desired state of the system has the added benefits of version control, audit and collaboration. These benefits were exemplified when CERN migrated the internal CMS project from a VM cluster to Kubernetes, as described in the two studies they published. The first study conducted by Imran et al. describes how the Kubernetes migration enhanced the overall system management compared to using a VM cluster [26]. While many of their experienced benefits stem from adopting containers instead of VMs,

the study still suggests Kubernetes provided many advantages, such as reduced operational costs. The other study conducted by Badran presents the practical benefits they experienced by using Kubernetes and is more focused on the adoption of a declarative approach for system management [27]. It describes how using Kubernetes instead of a VM cluster allowed them to craft reusable and composable configurations that enabled their entire infrastructure to be managed as code.

The benefits of moving into Kubernetes are further exemplified in a study by Gelley [28]. It presents a case study where an organization moved its business application from a Platform as a Service (PaaS) provided by Cloud Foundry into Kubernetes. The study not only describes how Kubernetes provided tremendous flexibility in terms of configuration and deployment compared to the previous PaaS but also how the organization was able to leverage the Kubernetes ecosystem to provide additional features such as auto-scaling and monitoring. Moreover, by moving into Kubernetes, possible future migration to other platforms was made simpler due to how widely adopted Kubernetes is.

User experience in Kubernetes was investigated in a study by Packard et al. that gathered several scenarios in which projects were built and managed using Kubernetes [29]. Many aspects were considered in the study, including administration such as managing persistent storage and limiting user resources with quotas, but also developer experience such as building and deploying applications. The study found that while system administrators face a learning curve when using Kubernetes, such as understanding the Kubernetes API and the ecosystem, users without prior knowledge of Kubernetes were able to successfully deploy applications. This finding is important to consider when evaluating KubeVirt, as it suggests that KubeVirt is a viable option even for users without prior knowledge of Kubernetes. Another study that investigated the user experience of Kubernetes was conducted by Boronin [30]. The study presents a case study where an organization faced a migration from a traditionally monolithic system architecture to a more cloud-native design. The study is relevant as it not only describes the challenges or benefits of Kubernetes itself but also considers the organizational impact of migrating to Kubernetes. This involves transferring skill sets among personnel, and the impact on the organization's culture.

In summary, these studies suggest that the benefits of using Kubernetes as a platform are not directly tied to what kind of resources are managed, but rather to the platform itself. Thus, when evaluating KubeVirt, it is important to not only consider benefits of KubeVirt itself but also the benefits that follow by adopting Kubernetes.

3.4 Performance Evaluations

Regarding performance, several studies have evaluated popular systems that can manage VMs. For instance, open-source alternatives such as OpenStack and CloudStack, or proprietary systems such as Amazon EC2.

A study by Kang et al. compared the performance of OpenStack and Amazon EC2 using a wide range of configurations and analyzed diverse aspects based on different hypervisor, storage and networking configurations [31]. Apart from the performance comparisons, the study also investigated performance impact when using more advanced features, such as Infiniband networking instead of Ethernet. The study showed mixed results, where OpenStack outperforms EC2 in some scenarios such as when using Infiniband networking, but EC2 tends to outperform OpenStack in parallel processing. As such, the study concluded that component-wise, each system has strengths. This study brings with it relevant perspectives as it highlights many of the aspects that are important to consider when evaluating the performance of VM management systems.

Another study by Mullerikkal and Sastri compared OpenStack and CloudStack regarding imple-

mentation complexity, overall stability and performance [32]. The performance tests were designed to measure disk- and network I/O, and not CPU or memory usage. The study found that while OpenStack had a more complex implementation, it was more stable. Furthermore, by using a set of benchmarks it found that OpenStack had a better overall performance compared to CloudStack. However, a few years later, a complementary comparison between OpenStack and CloudStack was conducted by Paradowski et al. [33]. The comparison focused on performance and used several benchmarks such as deployment time, deletion time, and CPU utilization during deployment and deletion, and revealed significant and quantifiable differences between the two platforms. These tests were designed to represent user interactions with the system. However, the scale of the tests used in the comparison was relatively small and did not explore how the systems manage VMs under higher load. Analyzing the performance of a VM management system under higher load is important, as it can reveal how well a system scales when the ratio between VMs and nodes increases.

In contrast, a later study by Amaral benchmarked KubeVirt uses a setup with a large number of VMs relative to the size of the nodes, aiming to explore the limits of KubeVirt's control plane (scheduler etc.) [34]. The study found that, while KubeVirt was able to create all VMs, the control plane suffered a heavy load, indicating a possible large overhead when KubeVirt provisions a large number of VMs. This finding is important to consider when evaluating KubeVirt, as it indicates that KubeVirt may not yet be suitable for managing a large number of VMs.

In terms of network performance, a study conducted by Kolahdouzan investigated the networking traffic pattern in KubeVirt and OpenStack by performing networking performance tests to evaluate potential VM overhead [35]. The study found that the network overhead was significantly larger for scenarios in which VMs reside on separate nodes. However, while the study found overhead for both KubeVirt and OpenStack, it found that OpenStack had a considerably larger overhead. While this project will not focus on networking performance, it is important to consider the findings of this study when evaluating KubeVirt.

Hypervisors are a key component of VM management systems, and several studies have evaluated the performance of hypervisors. A study by Hwang et al. evaluated the performance of a range of hypervisors including KVM, Hyper-V and Xen [36]. The study found that applications and assigned resources had a significant impact on the overhead of the hypervisors. However, the study concluded that no single hypervisor was the silver bullet and that the choice of hypervisor should be based on the specific use case. Reviewing the performance of hypervisors is important when evaluating KubeVirt, as KubeVirt uses KVM as its hypervisor.

4 Research Questions

What performance, scalability, and operational efficiencies can be gained by integrating VMs in Kubernetes?

This could stem from using a single resource scheduler. Several metrics will be used to evaluate this, as described in the background and related work.

What are the current challenges and limitations in managing VMs and containerized applications in separate environments?

This is concerned with scenarios in which a traditional VM management system is operated alongside

Kubernetes.

How can system administration be simplified by running VMs in Kubernetes using KubeVirt?

This is focused on the practical implications of using KubeVirt and Kubernetes instead of traditional VM management systems. For instance, the number of steps required to deploy a VM.

5 Hypothesis

Managing virtual machines in Kubernetes significantly improves operational efficiency compared to traditional virtual machine management systems. The improvement is hypothesized to stem from accelerated deployment speed, easier management, simplified scalability, and improved resource utilization.

6 Tasks

1. Literature review
   Review existing literature on Kubernetes, KubeVirt and traditional virtual machine management systems, such as OpenStack, CloudStack and OpenNebula. The literature review should include feature explorations, benchmarks and evaluations.

2. Environment setup
   Establish a testing environment that can be used for exploration and feature evaluation. This includes Kubernetes with KubeVirt, OpenStack, OpenNebula, and CloudStack. The latter two are already present at Net Insight and kthcloud respectively, but there will still be laboratory setups in order to perform fair tests and benchmarks.

3. Research
   This includes the technologies behind KubeVirt and how deeply it integrates with Kubernetes.

4. Data collection
   Collect data regarding measurements for performance and resource utilization. This task includes crafting the necessary test environments, such as scripts to automate the deployment and deletion of VMs.

5. Data analysis
   Analyze the collected data to identify patterns, trends, and correlations between different variables, such as performance metrics and resource utilization in each environment. KPIs could be time to deploy a virtual machine, CPU usage, memory consumption, network usage when deploying/deleting a virtual machine etc.

6. Interview or supervised testing
   Perform some form of qualitative analysis, this could be an interview with some system administrators at Net Insight, or some form of observation when using KubeVirt; Does a system administrator experience KubeVirt as an improvement? The questions or tasks should be crafted to ensure they add value to the project.

7 Methods

- Quantitative analysis
  This is tied to *Data collection* and *Data analysis* in Tasks. It includes measuring performance, scalability and operational efficiencies that could arise when using Kubernetes with KubeVirt. Measured using metrics such as deployment speed, how quick scaling is, and how system resources are utilized (idle time, CPU usage per reserved CPU core) - essentially measuring the differences between orchestrating with traditional virtual machine systems and Kubernetes with KubeVirt.

  The test payload that will be used to measure performance will be crafted as a part of the method. It will consist of a set of scripts that will automate the entire testing process, including the deployment and deletion of VMs.

- Qualitative analysis
  This is tied to *Interview or supervised testing* in Tasks. It includes investigating the usability of a Kubernetes environment with KubeVirt. It will include collecting opinions and feedback from people using the system to answer what the perceived effect of using KubeVirt is. This could be system administrators at Net Insight.

- System Management
  This can be seen as a subsection of both Quantitative analysis and Qualitative analysis, as it emphasizes both usability as a qualitative metric as well as quantifiable metrics. It involves investigating possible simplifications of system management tasks, such as creating, deleting, starting, and stopping virtual machines etc.

8 Ethics and Sustainability

One interest in this project is comparing the hardware resource efficiency, which is indicated by the relation between how much work is required by the hardware and the amount of effective work that is done. This could mean that, given an environment that uses resources more efficiently (by moving to KubeVirt), less power could be needed.

Further, as the project also aims to unify environments, it could also mean more efficient use of servers. For instance, if different servers are dedicated to each environment before, a unified environment could share the physical hardware more evenly, thus requiring less hardware to be acquired.

9 Limitations

1. This project will not investigate any solutions for shortcomings in KubeVirt itself, for example, finding alternatives for features that exist in existing virtual machine management systems but not in KubeVirt. As such, this project acknowledges that KubeVirt might not be a complete replacement fr traditional virtual machine management tools (also described in [37]), as it might not offer all the features of an IaaS system. Therefore, only features that exist in KubeVirt and the compared system will be considered when testing.

2. From the description of this project above, there are also mainly two scenarios that will be evaluated.

- A system uses traditional virtual management (OpenNebula, CloudStack etc.).

  In this scenario, Kubernetes with KubeVirt is seen mainly as a 1:1 replacement.

- A system uses both traditional management and Kubernetes.

  This scenario is more aimed towards unifying the environments using KubeVirt.

10 Risks

This project will require both hardware and existing environments to properly evaluate the possible benefits of using KubeVirt. Acquiring hardware and asserting access to existing environments is crucial to ensure that all aspects of the project can be completed.

To avoid any issues with the aforementioned, I will consult my supervisor at Net Insight as early as possible with the required hardware to complete the project. I will also ensure that I am given access to any testing environments as early as possible.

11 Evaluation

Fulfillment of objects is determined by a combination of the results of qualitative feedback from Net Insight users (interviews, supervised testing) and quantitative measurements of performance and resource utilization.

The result of this project will contribute to the understanding of the integration of virtual machines in Kubernetes, as such a take is not widely spread. This means it could extend what is commonly associated with Kubernetes (containers and networking) to also include virtual machines. It will provide empirical data that could suggest, apart from possible benefits in administration and management, a measurable boost in performance and resource utilization. This research will be of interest to developers and IT professionals managing hybrid environments.

12 Conditions

This project will require physical servers and, preferably, access to virtual machines when exploring features myself. Both of which will be provided by Net Insight. This project also requires access to previous dual environments, one of which exists in Net Insight and one in KTH (kthcloud) (see Project proposal). These will be used as a base when doing before-and-after evaluations, and as a guide when building a transition roadmap (See Methods).

The external supervisor will ensure I am given the correct equipment, such as servers and virtual machines, and that I am not stuck due to, for instance, bad credentials. Since the supervisor is also responsible for Net Insight's current dual environment, he will guide them through their setup, possibly undocumented parts, and any unclear configuration they might use.

References

[1] Awodele Oludele et al. "On the evolution of virtualization and cloud computing: A review". In: *Journal of Computer Sciences and Applications* 2.3 (2014), pp. 40–43.

[2] Susanta Nanda Tzi-cker Chiueh and Stony Brook. "A survey on virtualization technologies". In: *Rpe Report* 142 (2005).

[3] M. Rosenblum and T. Garfinkel. "Virtual machine monitors: current technology and future trends". In: *Computer* 38.5 (2005), pp. 39–47. DOI: 10.1109/MC.2005.176.

[4] IBM. *Virtualization*. Feb. 2024. URL: https://www.ibm.com/topics/virtualization.

[5] Ankita Desai et al. "Hypervisor: A survey on concepts and taxonomy". In: *International Journal of Innovative Technology and Exploring Engineering* 2.3 (2013), pp. 222–225.

[6] Vitor Goncalves da Silva, Marite Kirikova, and Gundars Alksnis. "Containers for Virtualization: An Overview." In: *Appl. Comput. Syst.* 23.1 (2018), pp. 21–27.

[7] Dennis Gannon, Roger Barga, and Neel Sundaresan. "Cloud-native applications". In: *IEEE Cloud Computing* 4.5 (2017), pp. 16–21.

[8] Ramzi Debab and Walid Khaled Hidouci. "Containers runtimes war: a comparative study". In: *Proceedings of the Future Technologies Conference (FTC) 2020, Volume 2*. Springer. 2021, pp. 135–161.

[9] Xingyu Wang, Junzhao Du, and Hui Liu. "Performance and isolation analysis of RunC, gVisor and Kata Containers runtimes". In: *Cluster Computing* 25.2 (2022), pp. 1497–1513.

[10] OpenStack Foundation. *OpenStack*. Accessed 2024-03-06. Mar. 2024. URL: https://www.openstack.org/.

[11] Apache CloudStack. *Apache CloudStack*. Accessed 2024-03-06. Mar. 2024. URL: https://cloudstack.apache.org/.

[12] OpenNebula. *OpenNebula*. Accessed 2024-03-06. Mar. 2024. URL: https://opennebula.io/.

[13] Kubernetes. *Kubernetes*. Accessed 2024-03-06. Mar. 2024. URL: https://kubernetes.io/.

[14] HashiCorp. *Nomad*. Accessed 2024-03-06. Mar. 2024. URL: https://www.nomadproject.io/.

[15] Docker. *Docker Swarm*. Accessed 2024-03-06. Mar. 2024. URL: https://docs.docker.com/engine/swarm/.

[16] Guoqing Li et al. "The Convergence of Container and Traditional Virtualization: Strengths and Limitations". In: *SN Computer Science* 4.4 (May 2023). https://doi.org/10.1007/s42979-023-01827-9, p. 387. ISSN: 2661-8907. DOI: 10.1007/s42979-023-01827-9.

[17] The KubeVirt.io website CNCF. *Kubevirt.io*. Jan. 2024. URL: https://kubevirt.io/.

[18] Zou Nengren September 22, Ambassador post by Zou Nengren, and September 22. *Best practices for transitioning VM clusters to KubeVirt 1.0*. Sept. 2023. URL: https://www.cncf.io/blog/2023/09/22/best-practices-for-transitioning-vm-clusters-to-kubevirt-1-0/.

[19] Marcin Kubacki. *what is Kubevirt*. Feb. 2024. URL: https://storware.eu/blog/what-is-kubevirt/.

[20] Saumya Gupta et al. "Prevalence of GitOps, DevOps in Fast CI/CD Cycles". In: *2022 International Conference on Machine Learning, Big Data, Cloud and Parallel Computing (COM-IT-CON)*. Vol. 1. 2022, pp. 589–596. DOI: 10.1109/COM-IT-CON54601.2022.9850786.

[21] Pierre Le Fevre and Emil Karlsson. *Designing and implementing a private cloud for student and faculty software projects*. 2022.

[22] Pierre LeFevre and Emil Karlsson. *kthcloud*. URL: https://cloud.cbh.kth.se/.

[23] Supakorn Trakulmaiphol and Krerk Piromsopa. "An Implementation of DNS Operator and Network Guideline for Migrating Virtual Machine to KubeVirt". In: *2022 26th International Computer Science and Engineering Conference (ICSEC)*. 2022, pp. 84–89. DOI: 10.1109/ICSEC56337.2022.10049369.

[24] Mengmei Ye et al. "Free the Turtles: Removing Nested Virtualization for Performance and Confidentiality in the Cloud". In: *2023 IEEE 16th International Conference on Cloud Computing (CLOUD)*. 2023, pp. 275–281. DOI: 10.1109/CLOUD60044.2023.00039.

[25] Oskar Sonninen. "Perceived benefits of declarative software deployment: an exploratory case study". In: (2020).

[26] Muhammad Imran et al. "Migration of CMSWEB cluster at CERN to Kubernetes: a comprehensive study". In: *Cluster Computing* 24.4 (2021), pp. 3085–3099.

[27] Rukaiah NJH Badran. *Automation and improvement in CMSWEB Kubernetes clusters*. Tech. rep.

[28] Sravanthi Gelley. "Migrate Cloud Foundry Application to Kubernetes". In: (2022).

[29] Michael Packard et al. "Real-World, Self-Hosted Kubernetes Experience". In: *Practice and Experience in Advanced Research Computing*. 2021, pp. 1–5.

[30] Mikhail Boronin. *Hybrid Cloud Migration Challenges. A case study at King*. 2020.

[31] Mikyung Kang et al. "A comparison of system performance on a private openstack cloud and amazon ec2". In: *2017 IEEE 10th International Conference on Cloud Computing (CLOUD)*. IEEE. 2017, pp. 310–317.

[32] Jaison Paul Mullerikkal and Yedhu Sastri. "A comparative study of OpenStack and CloudStack". In: *2015 Fifth International Conference on Advances in Computing and Communications (ICACC)*. IEEE. 2015, pp. 81–84.

[33] Aaron Paradowski, Lu Liu, and Bo Yuan. "Benchmarking the Performance of OpenStack and CloudStack". In: *2014 IEEE 17th International Symposium on Object/Component/Service-Oriented Real-Time Distributed Computing*. 2014, pp. 405–412. DOI: 10.1109/ISORC.2014.12.

[34] Marcelo Amaral. *Kubevirt scale test by creating 400 vmis on a single node for FOSDEM 2022: IBM Research*. Feb. 2022. URL: https://research.ibm.com/publications/kubevirt-scale-test-by-creating-400-vmis-on-a-single-node.

[35] RISSO FULVIO GIOVANNI OTTAVIO and Alireza Kolahdouzan. "Network Automation, Orchestration, Cloud: Tools and Technologies". In: (2019).

[36] Jinho Hwang et al. "A component-based performance comparison of four hypervisors". In: *2013 IFIP/IEEE International Symposium on Integrated Network Management (IM 2013)*. IEEE. 2013, pp. 269–276.

[37] Parth Yadav and Vipin Kumar Rathi. "KupenStack: Kubernetes based Cloud Native OpenStack". In: *arXiv preprint arXiv:2106.02956* (2021).